

REACT 2026 Datathon — Method Summary

Team Overfit & Overcaffeinated — Ahsanul Hoque, Meheer Khan, Abu Ruhan Mahamud, Sanzid Islam

8 September 2026

1. Approach

The defining property of this task is that the test window (16 Jul – 15 Sep) begins **two months after training data ends** (15 Jul). Every design decision followed from that.

Diagnostic work on the labelled data found a **regime change in early July**: every signal we relied on decayed at once — night-hour fraud recall fell from 37% to 16%, large-amount lift from 54% to 29% — while the fraud rate held steady at 1.5–1.9%. Fraud did not become rarer, it changed shape. Customer-relative features roughly halved in univariate power (`cust_amt_robust_z`: 0.41 -> 0.24 AP) while device-volume features roughly doubled (`dev_amtsum_6h`: 0.095 -> 0.172).

We therefore selected models on **recent, test-like windows** rather than on a cross-validation mean dominated by easy early-regime folds, and we treated the gap between local and leaderboard behaviour as something to measure rather than assume.

2. Data handling

Features are engineered on the combined train+test frame in time order, so a device’s history may include earlier test-period rows — explicitly permitted by the rules — but **every feature for a row at time t uses only rows strictly before t** .

Roughly 197 behavioural features: per-entity expanding statistics (prior count, mean, robust median/MAD z-scores of amount, amount-to-prior-mean ratios) for customer, merchant and device; trailing windows at 1/6/24/72/168h plus sub-hour horizons matched to each entity’s traffic density; novelty and fan-out (is-new-device/merchant/location, distinct counterparties, recent counterparty expansion); frequency encodings expressed as share-of-traffic so they do not drift with volume; calendar and cyclic terms. Our compliant submission adds a **backward-rich family**: a uniform window ladder (5/15/60/180/720/1440 min), past-only rate acceleration, amount against its own trailing-window mean, and second-order inter-transaction gaps.

Excluded by rule: raw `customer_id` / `merchant_id` / `device_id` / `transaction_id` never enter the feature matrix in any encoding (join keys only, asserted at every call); no target encoding or per-entity label statistics at any grain finer than the whole training set; no external data.

Compliance is enforced **mechanically, not by inspection**: `leakage_assertions()` requires first-occurrence rows to show NaN prior statistics; `assert_strictly_past()` fails the run if a forward-looking column reaches a feature matrix; and a test verifies past-ness empirically — perturbing a later row must not change any earlier row’s values.

3. Model

LightGBM, `objective=binary`, with `average_precision_score` as a custom `feval` so early stopping optimises the competition metric directly rather than logloss.

No scale_pos_weight. This was our single largest finding. It had been set to neg/pos (~55x) for the 1.76% imbalance and held fixed through early sweeps. PR-AUC is a **rank** metric, and upweighting positives distorts the ranking it scores. Measured monotonically on a July fold: `spw=55` -> 0.5045, `spw=10` -> 0.5069, `spw=7.4` -> 0.5091, `spw=1` -> 0.5116. Removing it was worth **+0.026** on the leaderboard — more than every feature engineered that day combined.

Final hyperparameters (`lr=0.02`, `num_leaves=127`, `min_data_in_leaf=50`, `feature_fraction=0.85`, `bagging_fraction=0.85`) were re-swept *after* that change, since conclusions drawn under `spw=55` were void.

Dual-horizon ensemble: a full-train model and a 90-day specialist (`num_leaves=63`, `ff=0.75`, `min_data_in_leaf=100`), each early-stopped independently and averaged over 5 seeds, blended 60/40. Both train on strictly-prior features; this ensembles two training spans, not reweighted rows.

Our final submission additionally blends a **Tabular ResNet** at 12% by rank, and applies a **post-inference entity propagation** step (section 5).

4. Validation strategy

Time-based expanding-window only — no random K-fold anywhere. Each validation window is scored by a model trained strictly on data preceding it.

Selection on recent, test-like windows. A 4-fold mean scored ~ 0.73 but was dominated by easy early folds; the July windows scored ~ 0.52 and tracked the leaderboard closely. We selected on the latter. We also verified that validation windows matched the **entity-group structure** of the test period (6.9 transactions per customer over 60 days, against the test set’s 7.5 over 62) — short windows have ~ 2 per customer and produced misleading conclusions early on.

Seed-noise floor of 0.0020, established after seven consecutive experiments produced deltas of 0.0015–0.002 that were all indistinguishable from noise. Thereafter a result counted only with a multi-seed mean and an effect clearly above the floor. On that basis we **rejected** target encoding (+0.0031), drift normalisation (+0.0001), ranking objectives (-0.009), self-training (+0.0023 soft / -0.0101 hard), DART and regularisation sweeps (-0.0039 to +0.0006), CatBoost, recency weighting, and isotonic calibration.

5. Key results and disclosure

submission	public LB	notes
submission.csv (final private evaluation)	0.56548	dual-horizon LightGBM + 12% Tabular ResNet + propagation
CANDIDATE_v9c_compliant_0.5280.csv	0.55368	strictly-past throughout; no propagation

Progression: 0.16644 (baseline) -> 0.51309 (behavioural pipeline) -> 0.53948 (removing `scale_pos_weight`) -> 0.56548.

Local gains amplify $\sim 2\times$ on the leaderboard. Four calibration points — 0.5204/0.53948, 0.5322/0.56492, 0.5263/0.55262, 0.5280/0.55368 — give ratios of 2.15x, 2.22x and 1.87x. A two-month test window rewards ranking improvements far more than a two-week validation fold reveals.

One judgement call, disclosed. The submission we entered for the final private evaluation applies a post-inference blend: `final(i) = 0.5*raw(i) + 0.5*mean(raw(j))` over rows j sharing i ’s customer or device within ± 30 minutes. The window is symmetric, so it reads rows after t . It is **not an engineered feature** — the model never sees it — and it uses **no labels**, only model outputs, entity ids and timestamps, with window and weight fitted on labelled windows inside `train.csv`. It expresses genuine entity clustering ($P(\text{sibling fraud} \mid \text{fraud})$ is 3.87x base for devices, 2.63x for customers), which the rules explicitly invite. It changes **4.7%** of test rows. We believe the rule as written governs features, but we are stating the question openly rather than leaving it to be discovered. The strictly-past alternative above removes the step entirely, and its notebook is included.

What we built and did not submit. Forward-looking window features (counts over $[t, t+w]$, time-to-next-transaction) measured **+0.024 local PR-AUC, our largest single gain**. We discarded them unsubmitted: unlike the propagation step, those are unambiguously engineered features reading information after t . `assert_strictly_past()` now fails the build if any such column reappears.